

Task 6: Automated Logging using Triggers & Views

A trigger that logs every INSERT or UPDATE A view that shows daily activity reports Real-Time Usage: Audit logging in enterprise databases

AIM:

To create a database trigger that automatically logs every INSERT or UPDATE operation performed on a table, and to create a view that summarizes daily activity logs for auditing purposes.

ALGORITHM:

Step 1 : Create a main table (e.g., employees).

Step 2 :Create a log table (e.g., employees_log) to store action details (who, when, what, etc.).

Step 3 :Create a trigger that fires AFTER INSERT or UPDATE on the main table:

- Captures the operation type.
- Inserts a record into the log table with date, time, operation type, and data details.

Step 4 :Create a view (daily_activity_report) that summarizes the count of inserts and updates by date.

Step 5 :Execute test DML operations (insert/update).

Step 6 :Query the view to verify daily summary.

PROGRAM:

SQL

```
-- Create Database
```

```
CREATE DATABASE audit_lab;
```

```
USE audit_lab;
```

1. Create Main Table

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary DECIMAL(10,2),  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
    ON UPDATE CURRENT_TIMESTAMP  
);
```

2. Create Log Table

```
CREATE TABLE employees_log (  
    log_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_id INT,  
    action_type VARCHAR(10),  
    action_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
    old_salary DECIMAL(10,2),  
    new_salary DECIMAL(10,2)  
);
```

3. Trigger for INSERT

```
DELIMITER $$  
  
CREATE TRIGGER trg_employees_insert  
AFTER INSERT ON employees  
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employees_log(emp_id, action_type, new_salary)
```

```
    VALUES (NEW.emp_id, 'INSERT', NEW.salary);
```

```
END $$
```

```
DELIMITER ;
```

4. Trigger for UPDATE

```
DELIMITER $$
```

```
CREATE TRIGGER trg_employees_update
```

```
AFTER UPDATE ON employees
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employees_log(emp_id, action_type, old_salary, new_sal-  
    ary)
```

```
    VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary);
```

```
END $$
```

```
DELIMITER ;
```

5. Create View

```
CREATE VIEW daily_activity_report AS
```

```
SELECT
```

```
    DATE(action_time) AS action_date,
```

```
    action_type,
```

```
    COUNT(*) AS total_actions
```

```
FROM employees_log
```

```
GROUP BY DATE(action_time), action_type
```

```
ORDER BY action_date;
```

6. Insert Records

```
INSERT INTO employees (name, department, salary)
VALUES ('Alice', 'HR', 40000),
('Bob', 'IT', 50000);
```

7. Update Record

```
UPDATE employees
SET salary = 52000
WHERE name = 'Bob';
```

8. Check Logs

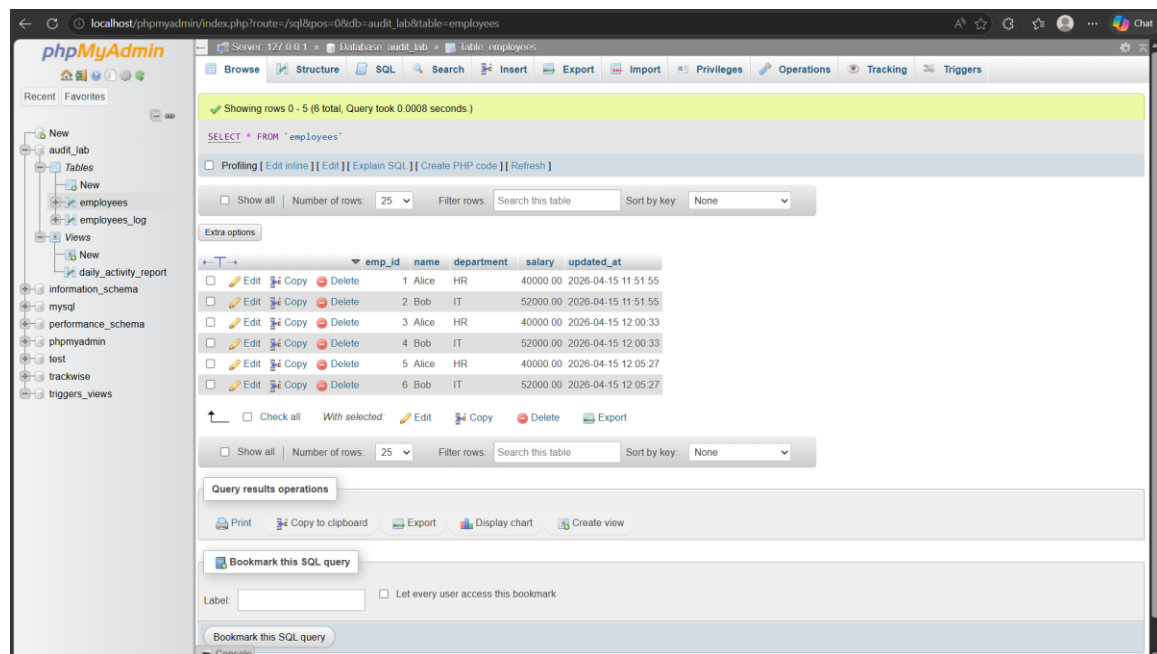
```
SELECT * FROM employees_log;
```

9. View Daily Report

```
SELECT * FROM daily_activity_report;
```

OUTPUT:

TRIGGERS



The screenshot shows the phpMyAdmin interface for a database named 'audit_lab'. The 'employees' table is selected, and the 'Table: employees' view is active. The table contains 6 rows of data. The interface includes a sidebar with a tree view of the database structure, a top navigation bar with tabs like 'Browse', 'Structure', 'SQL', 'Search', 'Insert', 'Export', 'Import', 'Privileges', 'Operations', 'Tracking', and 'Triggers'. The main area displays the table data with columns: emp_id, name, department, salary, and updated_at. Below the table, there are options to 'Check all', 'With selected', 'Edit', 'Copy', 'Delete', and 'Export'. At the bottom, there is a 'Query results operations' section with links for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. A 'Bookmark this SQL query' section is also present, allowing users to save the current query for later use.

emp_id	name	department	salary	updated_at
1	Alice	HR	40000.00	2026-04-15 11:51:55
2	Bob	IT	52000.00	2026-04-15 11:51:55
3	Alice	HR	40000.00	2026-04-15 12:00:33
4	Bob	IT	52000.00	2026-04-15 12:00:33
5	Alice	HR	40000.00	2026-04-15 12:05:27
6	Bob	IT	52000.00	2026-04-15 12:05:27

Showing rows 0 - 11 (12 total, Query took 0.0006 seconds)

```
SELECT * FROM `employees_log`
```

Number of rows: 25 | Filter rows: Search this table | Sort by key: None

	log_id	emp_id	action_type	action_time	old_salary	new_salary
☐	1	1	INSERT	2026-04-15 11:51:55	NULL	40000.00
☐	2	2	INSERT	2026-04-15 11:51:55	NULL	50000.00
☐	3	2	UPDATE	2026-04-15 11:51:55	50000.00	52000.00
☐	4	3	INSERT	2026-04-15 12:00:33	NULL	40000.00
☐	5	4	INSERT	2026-04-15 12:00:33	NULL	50000.00
☐	6	2	UPDATE	2026-04-15 12:00:33	52000.00	52000.00
☐	7	4	UPDATE	2026-04-15 12:00:33	50000.00	52000.00
☐	8	5	INSERT	2026-04-15 12:05:27	NULL	40000.00
☐	9	6	INSERT	2026-04-15 12:05:27	NULL	50000.00
☐	10	2	UPDATE	2026-04-15 12:05:27	52000.00	52000.00
☐	11	4	UPDATE	2026-04-15 12:05:27	52000.00	52000.00
☐	12	6	UPDATE	2026-04-15 12:05:27	50000.00	52000.00

Query results operations: Copy to clipboard, Export, Display chart, Create view

VIEWS

Showing rows 0 - 1 (2 total, Query took 0.0009 seconds)

```
SELECT * FROM `daily_activity_report`
```

Number of rows: 25 | Filter rows: Search this table

action_date	action_type	total_actions
2026-04-15	INSERT	6
2026-04-15	UPDATE	6

Query results operations: Print, Copy to clipboard, Export, Display chart, Create view

Bookmark this SQL query

Label: ☐ Let every user access this bookmark

Bookmark this SQL query

RESULT:

A trigger successfully logs every INSERT and UPDATE operation in the employees_log table. The daily_activity_report view provides daily summarized activity for auditing and monitoring.